Fitness and Diet Tracker with Personalized Recommendations

ARCHITECTURE DESIGN ANALYSIS REPORT

Submitted in the partial fulfilment for the Course of

CSA1012-SOFTWARE ENGINEERING

to the award of the degree of

BACHELOR OF ENGINEERING

IN

INFORMATION TECHNOLOGY

Submitted by

K.ABISHEK (192521120)

Under the Supervision of

Dr.K.ANITA DAVAMANI

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

AUGUST 2026

1. ANALYSIS OF SYSTEM REQUIREMENTS

1.1 System Objectives

- ➢ Develop a personalized fitness and diet management platform for individual users.
- ➢ Continuously track daily physical activity and nutritional intake.
- ➢ Generate AI-based, individualized workout and diet recommendations.
- ➢ Integrate with wearable devices for automatic health-metric synchronization.
- ➢ Visualize user progress through interactive dashboards and reports.
- ➢ Deliver timely reminders for exercise, meals, and hydration.
- ➢ Ensure secure storage and processing of sensitive personal health data.
- ➢ Promote sustained, healthy lifestyle habits through continuous monitoring.

1.2 Functional Requirements Influencing Architecture

The system must support user profile management, daily activity and nutrition logging, an AI/ML-based recommendation engine for workouts and diet plans, automatic wearable-device synchronization, dashboards and exportable progress reports, and a reminder/notification service. These requirements imply a system that (a) ingests continuous, high-frequency telemetry from external wearable APIs, (b) runs computationally distinct AI recommendation logic separate from simple CRUD operations, and (c) must push time-sensitive notifications independently of user-initiated requests — all of which push the design toward a component structure with clearly separated concerns rather than a single monolithic codebase.

1.3 Non-Functional Requirements Influencing Architecture

Quality Attribute	Requirement Driving the Architecture
Scalability	Must support at least 10,000 concurrent users and scale horizontally as the user base grows.
Performance	Dashboard updates within 5 seconds of new data; recommendations generated within 3 seconds.
Reliability	99.5% uptime with automatic failover for critical tracking and reminder services.
Security	TLS 1.2+ encryption in transit, role-based access control over sensitive health data.
Maintainability	Independent components (recommendation engine, tracking engine, dashboard) must be updatable without full-system redeployment.
Availability	Core tracking, logging, and reminder services must remain available 24/7, including during maintenance of non-critical modules.

Quality Attribute	Requirement Driving the Architecture
Interoperability	Must expose REST APIs to third-party wearable platforms (Fitbit, Google Fit, Apple Health) and nutrition databases.
Portability	Web dashboard must run on major browsers; mobile app must support Android and iOS.

1.4 Key Quality Attributes Identified

From the above analysis, six quality attributes emerge as the primary architectural drivers: **scalability** (growing user base and continuous telemetry), **maintainability** (independently evolvable recommendation logic and integrations), **reliability** and **availability** (uninterrupted tracking and reminders), **performance** (near real-time dashboard and recommendation response), and **security** (protection of sensitive personal health data). These six attributes directly shape the choice of architectural style in Section 2.

2. SELECTED SOFTWARE ARCHITECTURE

2.1 Proposed Architectural Style

A **hybrid Layered + Microservices architecture** is proposed for the Fitness and Diet Tracker platform. The system is first organized into three horizontal layers — **Presentation**, **Application/ Service**, and **Data** — and the Application/Service layer is further decomposed into independently deployable **microservices** (User Service, Activity Tracking Service, Nutrition Service, AI Recommendation Service, Wearable Integration Service, and Notification Service), coordinated through an API Gateway and an asynchronous event bus.

2.2 Justification for the Selected Architecture

➢ **Independent scalability:** The AI Recommendation Service is computationally heavier than the User or Notification services; microservices allow it to be scaled independently rather than scaling the entire application.

➢ **Separation of concerns:** Activity tracking, nutrition logging, recommendation generation, wearable synchronization, and notifications are distinct business capabilities with different data patterns (high-frequency telemetry vs. low-frequency profile edits) — a layered decomposition inside each service keeps this clean.

➢ **Maintainability and independent deployment:** NFR explicitly requires that components such as the recommendation engine be updatable without full-system redeployment — a core benefit of microservices.

➢ **Resilience/availability:** If the AI Recommendation Service or an external wearable API is temporarily unavailable, core tracking and reminder services can continue operating, satisfying the 24/7 availability requirement for critical modules.

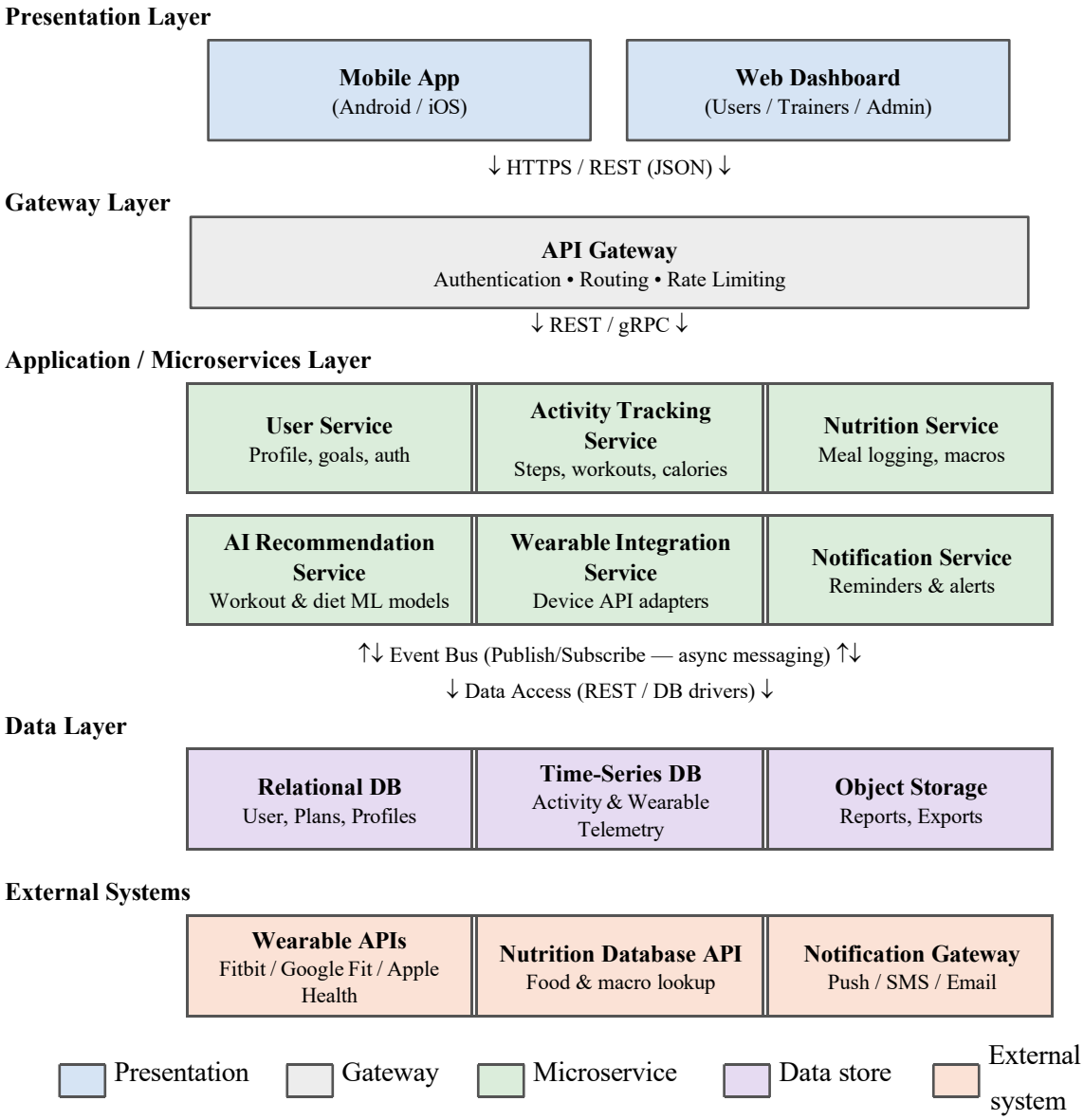
➢ **Third-party integration:** Wearable-device and nutrition-database integrations are naturally isolated into a dedicated Wearable Integration Service, limiting the blast radius of any external

API failure or rate-limiting.

- **Event-driven elements for real-time behaviour:** Reminders/notifications and wearable-sync events are naturally asynchronous; an event bus (publish/subscribe) between services supports this without tightly coupling services to one another.

A pure monolithic layered architecture was considered but rejected because it would force the AI recommendation workload, high-frequency telemetry ingestion, and simple user-management logic to scale and deploy together, working against the scalability and maintainability requirements. A pure microservices-only design (without an internal layered structure per service) was also rejected, as it would add unnecessary operational complexity for what remains a moderately sized application. The hybrid approach keeps the simplicity of layering within each service while gaining the scalability and independent-deployability benefits of microservices where they matter most.

3. SOFTWARE ARCHITECTURE DIAGRAM



Notation: Solid vertical arrows indicate synchronous REST/gRPC request-response calls between layers. The bidirectional arrow at the microservices layer indicates asynchronous publish/subscribe messaging over an event bus, used for cross-service events such as "activity logged," "wearable data synced," and "reminder due."

4. COMPONENTS AND THEIR INTERACTIONS

4.1 Component Responsibilities

Component	Responsibility
Mobile App / Web Dashboard	Presents activity, nutrition, and progress data to users; captures manual logs and profile edits; displays AI recommendations.

Component	Responsibility
API Gateway	Single entry point for all client requests; handles authentication, request routing to the correct microservice, and rate limiting.
User Service	Manages user profiles, fitness goals, dietary preferences, authentication, and role-based permissions.
Activity Tracking Service	Records and stores daily activity data (steps, workout duration, calories burned) from manual entry or synced devices.
Nutrition Service	Records meal logs, calculates calories and macronutrients, and maintains the nutrition history.
AI Recommendation Service	Runs machine-learning models to generate personalized workout and diet plans based on user profile, activity, and nutrition history.
Wearable Integration Service	Adapts and normalizes data from external wearable-device APIs into the platform's internal data format.
Notification Service	Evaluates trigger conditions (scheduled workout, hydration interval, missed target) and dispatches reminders/alerts.
Event Bus	Carries asynchronous events between microservices (e.g., activity logged, plan generated, reminder due) without direct coupling.
Relational Database	Stores structured, low-frequency data: user profiles, workout/diet plans, roles.
Time-Series Database	Stores high-frequency activity and wearable telemetry optimized for time-based queries and trend analysis.
Object Storage	Stores generated reports and exported files (CSV/PDF).

4.2 Communication Mechanisms

➢ Clients (mobile app, web dashboard) communicate with the backend exclusively through the API Gateway over HTTPS/REST, using JSON payloads.

➢ The API Gateway routes authenticated requests synchronously to the appropriate microservice over internal REST or gRPC calls.

➢ Microservices publish domain events (e.g., “ActivityLogged”, “WearableDataSynced”, “PlanGenerated”) to a shared event bus; interested services subscribe to relevant events asynchronously.

➢ The Notification Service subscribes to activity, nutrition, and reminder-related events and pushes alerts through the external Notification Gateway (push/SMS/email).

➢ The Wearable Integration Service polls or receives webhooks from external wearable APIs, normalizes the data, and publishes it to the event bus for consumption by the Activity Tracking

Service.

➢ Each microservice owns its own data access to the Relational or Time-Series database, avoiding direct cross-service database access.

4.3 Overall Workflow (Illustrative)

A typical end-to-end flow begins when a user's wearable device generates new activity data. The Wearable Integration Service retrieves this data via the external API, normalizes it, and publishes an "ActivityDataReceived" event on the event bus. The Activity Tracking Service consumes this event and updates the time-series activity log, which in turn triggers a "ActivityLogged" event. The AI Recommendation Service consumes relevant activity and nutrition events to periodically recalculate the user's workout and diet plan, storing the updated plan in the relational database and emitting a "PlanGenerated" event. The Notification Service listens for goal-related and reminder-related events and dispatches timely alerts through the Notification Gateway. Meanwhile, the user's mobile app or web dashboard queries the API Gateway on demand to render the latest activity, nutrition, and recommendation data pulled from the relevant services.

5. DISCUSSION OF QUALITY ATTRIBUTES

Attribute	How the Architecture Addresses It
Scalability	Each microservice can be scaled independently — e.g., the AI Recommendation Service can run more instances during peak recommendation-generation periods without scaling the User or Notification services, supporting the 10,000-concurrent-user target.
Security	The API Gateway centralizes authentication and enforces TLS 1.2+ and role-based access control before any request reaches an internal service, limiting the attack surface and keeping sensitive health data access consistently governed.
Performance	Time-series-optimized storage for high-frequency telemetry and asynchronous event processing keep the write path fast, while the API Gateway and service boundaries allow caching and horizontal scaling to meet the 5-second dashboard and 3-second recommendation targets.
Reliability	Failure in one microservice (e.g., a wearable API outage reaching the Wearable Integration Service) does not directly crash other services; the event bus buffers messages, allowing services to catch up once restored, supporting the 99.5% uptime target.
Maintainability	Each microservice can be developed, tested, and redeployed independently by different teams or at different times, directly satisfying the requirement that components be updatable without full-system redeployment.
Availability	Core services (Activity Tracking, Notification) are decoupled from the more resource-intensive AI Recommendation Service; if the recommendation model needs maintenance, core tracking and reminders continue operating uninterrupted.

6. JUSTIFICATION OF ARCHITECTURAL DECISIONS

6.1 Rationale Summary

The hybrid Layered + Microservices style was chosen because the problem statement combines two distinct workloads — simple, transactional CRUD operations (profile, logging) and computationally heavier, independently evolving AI logic (recommendations) — alongside a hard requirement for independent-component maintainability. A single architectural style applied uniformly (e.g., a pure layered monolith) would satisfy simplicity but not the explicit scalability and independent-deployment requirements; a pure microservices style without internal layering would satisfy scalability but add avoidable complexity to simpler services such as User Service. The hybrid approach balances both concerns.

6.2 Trade-offs Considered

- **Complexity vs. scalability:** Microservices introduce operational overhead (service discovery, distributed monitoring, network latency between services) compared to a monolith; this was accepted because the scalability and independent-maintainability requirements are explicit and high-priority.
- **Consistency vs. availability:** Using an event bus for cross-service updates means some data (e.g., a freshly generated recommendation) is eventually consistent rather than instantly consistent everywhere; this trade-off was accepted since near real-time (not strictly instantaneous) updates are sufficient for a fitness/diet use case.
- **Development speed vs. long-term flexibility:** A monolith would have been faster to build initially, but the hybrid architecture was chosen to avoid costly re-architecture later as the user base and AI model complexity grow.

6.3 Support for Future Enhancements and Long-Term Maintainability

The chosen architecture supports future growth in several ways. New capabilities — such as a telemedicine integration, a social/community feature, or a supplement marketplace — can be added as new microservices behind the existing API Gateway without modifying existing services. The AI Recommendation Service can evolve its underlying models (e.g., adopting more advanced ML techniques) independently, since it communicates with the rest of the system only through well-defined events and APIs. New wearable-device providers can be supported by extending the Wearable Integration Service alone. Because each service owns its own data store, teams can also introduce new storage technologies for a specific service (e.g., a specialized vector database for future AI features) without impacting the rest of the platform. This isolation of change is the core long-term maintainability benefit the hybrid architecture is intended to deliver.